

Extending Memory Space with Block RAM

Objectives

After completing this lab, you will be able to:

- Add BRAM and connect it to the processing system's AXI master port
- Execute the software application having data section in the BRAM

Steps

Open the Project

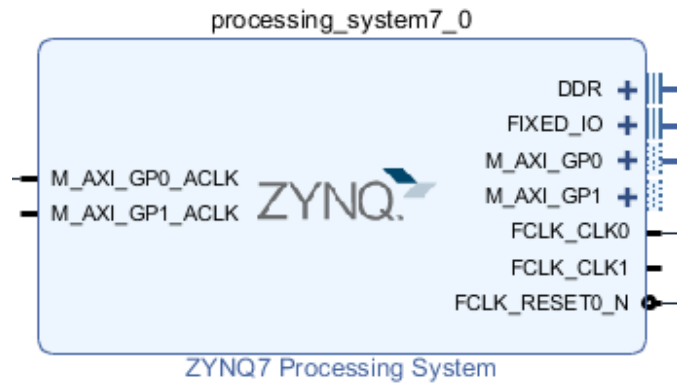
Open the Vivado program. Open the *lab1* project you created earlier or use the *lab1* project from the labsolution directory, and save the project as *lab3*.

1. Start Vivado if necessary and open either the *lab1* project (lab1.xpr) you created earlier or the lab1 project in the *labsolutions* directory using the **Open Project** link in the Getting Started page.
2. Select **File > Project > Save As ...** to open the *Save Project As* dialog box. Enter **lab3** as the project name. Make sure that the *Create Project Subdirectory* and *Import All Files to the New Project* options are checked, the project directory path is **{labs}** and click **OK**.

This will create the *lab3* directory and save the project and associated directory with lab3 name.

Configure the Processor to Enable M_AXI_GP1

1. Open the Block Design and enable the M_AXI_GP1 interface.
2. Click **Open Block Design** in the *Flow Navigator* pane
3. Double-click on the *Zynq processing system* instance to open its configuration form.
4. Select *PS-PL Configuration* in the Page Navigator window in the left pane, expand *AXI Non Secure Enablement>GP Master AXI Interface*, and click on the check-box of the **M_AXI GP1 Interface** to enable it.
5. Select *Clock Configuration* in the Page Navigator window in the left pane, expand *PL Fabric Clocks* on the right, and click on the check-box of the **FCLK_CLK1** to enable it.
6. Enter the *_Requested Frequency_* for the **FCLK_CLK1** as **140.00000** MHz.
7. Click **OK** to accept the settings and close the configuration form.

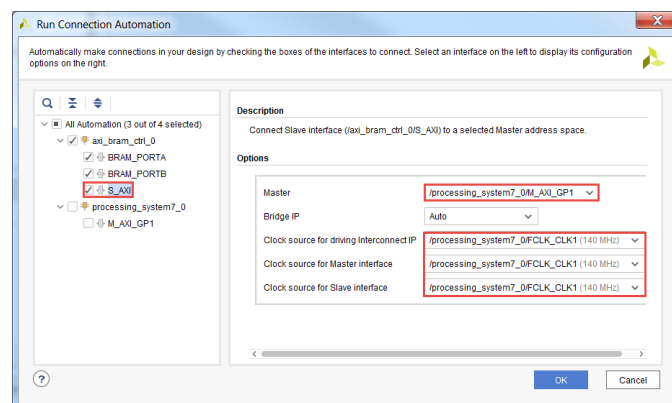


M_AXI_GP1 interface enabled

Extend with BRAM

Add an AXI BRAM Controller instance with BRAM.

1. Click the **+** button and search for **BRAM** in the catalog.
2. Double-click the **AXI BRAM Controller** to add an instance to the design.
3. Click on **Run Connection Automation**, and select **axi_bram_ctrl_0**
4. Click on **BRAM_PORTA** and **BRAM_PORTB** check boxes.
5. Click **S_AXI**, and change the *Master* option to **/processing_system7_0/M_AXI_GP1**, change the Clock source for driving interconnect IP, Clock source for Master interface, and *Clock source for Slave interface* to **/processing_system7_0/FCLK_CLK1 (140 MHz)** as they all run in the same clock domain, and click **OK**



Connecting AXI BRAM Controller to M_AXI_GP1 to run at faster clock speed

Notice that an instance of AXI SmartConnect and Processor System Reset are added, and the M_AXI_GP1_ACLK is connected to FCLK_CLK1.

Clocking network connections

Notice that the address range changed to 0x80000000-0x8000FFFF. This is in the M_GP1 addressing space.

3. Select the *Diagram* tab, and click on the  (Validate Design) button to make sure that there are no errors.

Generate the Bitstream

1. Click on the **Generate Bitstream** to run the synthesis, implementation, and bit generation processes.
2. Click **Save** if prompted to save the project, and **Yes** to run the processes. Click **OK** to launch the runs.
3. When the bitstream generation process has completed successfully, click **Cancel**.

Generate Applications in the SDK

Export the implemented design, and start SDK

1. Export the hardware configuration by clicking **File > Export > Export Hardware...**
2. Click the box to *Include Bitstream_and* and click **OK** (Click *_Yes* if prompted to overwrite the previous module)
3. Launch SDK by clicking **File > Launch SDK** and click **OK**
4. Right-click on the **lab1** and **standalone_bsp_0** and **system_wrapper_hw_platform_0** projects in the Project Explorer view and select **close project**.

Create an empty application project, named lab3, and import the provided lab3.c file.

1. Select **File > New > Application Project**.
2. In the *Project Name* field, enter **lab3** as the project name.
3. Use the default settings to create a new BSP and click **Next**.
4. Select the **Empty Application** template and click **Finish**.

The lab3 and lab3_bsp projects will be created in the Project Explorer window of SDK.

5. Select **lab3 > src** directory in the project view, right-click, and select **Import**.
6. Expand the **General** category and double-click on **File System**.
7. Browse to **{sources}\lab3** folder.
8. Select **lab3.c** and click **Finish**.

A snippet of the source code is shown in the following figure. It shows that we write a pattern to the LED port and execute a software delay loop. Repeat for 16 times. It also shows the code (greyed) which will be used in Lab5.

```

#include "xparameters.h"
#include "xgpio.h"
#ifdef MULTIBOOT
#include "xdevcfg.h"
#endif
//=====
int main (void)
{
    XGpio leds;
    int j=0;
    int i;
    xil_printf("-- Start of the Program --\n\n");
    XGpio_Initialize(&leds, XPAR_LEDS_DEVICE_ID);
    XGpio_SetDataDirection(&leds, 1, 0); // output
    for(j=0; j<16; j++) {
        XGpio_DiscreteWrite(&leds, 1, j);
        for (i=0; i<999999999; i++);
    }
    xil_printf("End of the program\n\n");
#ifdef MULTIBOOT
    print("Loading master image\n\n");
    // Driver Instantiations
    Xdcfg_XDcfg_0;
    u32 MultiBootReg = 0;
    #define PS_RST_CTRL_REG (XPS_SVS_CTRL_BASEADDR + 0x200)
    #define PS_RST_MASK 0x1 /* PS software reset */
    #define SLCR_UNLOCK_OFFSET 0x08

    // Initialize Device Configuration Interface
    XDcfg_Config *Config = XDcfg_LookupConfig(XPAR_XDCFG_0_DEVICE_ID);
    XDcfg_CfgInitialize(&XDcfg_0, Config, Config->BaseAddr);

    MultiBootReg = 0; // Once done, boot the master image stored at 0xfc00_0000
    Xil_Out32(0xf8000000 + SLCR_UNLOCK_OFFSET, 0xf000f000); // unlock SLCR
    XDcfg_WriteReg(XDcfg_0.Config.BaseAddr, XDCFG_MULTIBOOT_ADDR_OFFSET, MultiBootReg); // write to multiboot reg
    // synchronize
    __asm__(
        "dsb\n\t"
        "isb\n\t"
    );
    // Generate soft reset
    Xil_Out32(PS_RST_CTRL_REG, PS_RST_MASK);
#endif
    return 0;
}

```

Source Code

Test in Hardware

Connect and power up the board. Establish the serial communication using the SDK Terminal tab. Program the FPGA.

1. Connect and power up the board.
2. In SDK, select **Xilinx > Program FPGA** and click the **Program** button to program the FPGA.
3. Select the  **Terminal** tab. If it is not visible then select **Window > Show view > Terminal**.
4. Click on  to initiate the serial connection and select the appropriate COM port (depending on your computer). Configure it with 115200 baud rate.

Run the lab3 application.

1. Select the **lab3** project in *Project Explorer*, right-click and select **Run As > Launch on Hardware (System Debugger)**. Click **Yes** to terminate the previous run.

The application (lab3.elf) will be downloaded into the target device, execute ps7_init, and execute.

2. You should see the on-board LEDs changing patterns at roughly a one second delay rate.

Modify the linker script to use the ps7_ddr_0 for the code and data sections, and the BRAM for the Heap and Stack segments. Change the loop limit from 999999999 to 999999. Execute the program.

1. Select the **lab3** application in the *Project Explorer* view.
2. Right-click and select **Generate Linker Script**.

3. Change the *code* and *Data* sections to **ps7_dds_0** and the *Heap and Stack* segment memory to **axi_bram_ctrl_0_Mem0**.
4. Click the **Generate** button.
5. Click the **Yes** button to overwrite.
6. Change the loop limit from 99999999 to **9999999**. Save changes so the program recompiles.
7. Select the **lab3** project in *Project Explorer*, right-click and select **Run As > Launch on Hardware (System Debugger)**.

Click OK to terminate the existing run and relaunch if shown.
8. You should see the on-board LEDs changing patterns very slowly (about 5 seconds).
9. Change the loop limit from 99999999 to **9999999**. Save changes so the program recompiles.
10. Select the **lab3** project in *Project Explorer*, right-click and select **Run As > Launch on Hardware (System Debugger)**.

Click Yes to terminate the existing run.
11. You should see the on-board LEDs changing patterns relatively faster (about 1 seconds).
12. Close the SDK program by selecting **File > Exit**.
13. Close the Vivado program by selecting **File > Exit**.
14. Turn OFF the power on the board.

Conclusion

This lab led you through adding BRAM memory in the PL section thereby extending the total memory space available to the PS. You have verified the functionality by creating an application, targeting the stack and heap sections to the added BRAM, and executing the application.